

SQL Server Always On AG Failover POC on AWS — Implementation Plan

Context

This POC demonstrates SQL Server Always On Availability Group (AG) failover — continuing from earlier animated architecture diagrams (App → AG Listener → WSFC + 3 replicas) and PowerPoint decks built for this project. The goal is a working system, not just a diagram: a .NET backend + Angular UI that lets the user send read/write requests, watch live which SQL node is PRIMARY vs SECONDARY, run a load test, trigger a failover, and watch the app keep working while a secondary is promoted.

Constraints that shape every decision below: the user is a .NET backend developer with **no prior AWS or WSFC/AG experience** (first infrastructure deployment ever), has only **\$100 in AWS credit**, knows a little **Terraform** (chosen IaC tool), and wants both the .NET API and Angular UI **actually deployed** to AWS, not run locally.

Key architecture decisions (deviations from "textbook" production design, deliberate for cost/simplicity)

Decision	Why
Workgroup WSFC cluster, no Active Directory (domain-independent AG, cert-based endpoint auth)	AWS Managed AD ≈ \$88/month and can't be paused, only deleted. Microsoft officially supports domain-independent AGs since SQL 2016 — this is the single biggest cost/complexity saver.
No NAT Gateway — EC2/Fargate in public subnets, locked down via security groups	NAT Gateway ≈ \$33+/month and bills even while EC2 is stopped. Explicitly a POC-only shortcut, not production practice.
SQL Server Developer Edition via Chocolatey silent install	No AWS Marketplace AMI ships Developer Edition (only paid editions). Chocolatey's unattended install is a documented pattern.
NLB with TCP health check on a dedicated ProbePort (e.g. 59999) bound to the AG Listener cluster resource	This is AWS's own documented pattern (see sources in research). Only the node currently hosting the listener resource responds on that port — no custom polling service needed. NLB routes 1433 traffic only to whichever node passes that probe (i.e. current primary). ~20–40s failover detection.
.NET API on ECS Fargate, Linux containers	Windows Fargate costs ~2-3x Linux; .NET 8 runs natively on Linux.
One CloudFront distribution, two origins (S3 for Angular build, ALB for API/SignalR)	Avoids CORS and mixed-content issues; CloudFront passes through WebSockets for SignalR.

Decision	Why
Secrets in SSM Parameter Store (SecureString), not Secrets Manager	Free vs \$0.40/secret/month — fine for a POC.
Stop vs. destroy discipline is the main cost control	EBS volumes and load balancers keep billing while "stopped" — full teardown between sessions is the only way to hold costs near zero.
AWS Budget alert configured before any Terraform runs	Safety net against an accidental "left running overnight" mistake.

Cost picture (us-east-1, ballpark — verify in AWS Pricing Calculator before applying)

- 3× EC2 **t3.large** Windows (start here; bump to t3.xlarge only if SQL Server shows memory pressure during load tests) + NLB + ALB + Fargate + EBS ≈ **~\$2.50 per 4-hour active session**
- If left running/idle (not destroyed): EBS (~\$0.47/day) + NLB/ALB (~\$1.08/day if not deleted) ≈ **~\$46/month for a forgotten stack** — this is the main risk to the budget
- Reserve ~\$10–15 of the \$100 for the WSFC/AG bring-up phase, which will take 2–4 iterations. Remaining budget affords 30+ demo sessions if you tear down between them.
- Set AWS Budget alerts at \$20/\$50/\$80 in Phase 0, before touching Terraform.

Repo structure (under C:\Users\aaoyalata\OneDrive - Iskraemeco, d.d\AhmedData\FailOverSolution)

```
infra/
  envs/poc/          # main.tf, variables.tf, outputs.tf, terraform.tfvars, backend.tf
  modules/
    network/         # VPC, 3 public subnets, IGW, route tables, security groups
    sql-nodes/        # 3x EC2 Windows t3.large, EBS, IAM/SSM instance profile
    nlb-ag-listener/  # NLB, target group (TCP:1433, health check TCP:59999), listener
    api-fargate/      # ECS cluster/task/service, ALB, ECR repo, IAM roles, log group
    frontend-static/  # S3 (private, OAC) + CloudFront (2 origins: S3, ALB)
    observability/    # AWS Budget, CloudWatch log groups/dashboard
  scripts/
    bootstrap/        # install-sql-node.ps1, create-workgroup-cluster.ps1,
                      # configure-ag-certs.ps1, create-availability-group.ps1,
                      # configure-listener-probe.ps1
    ssm-documents/    # SSM Run Command/Automation docs wrapping the bootstrap scripts
    lifecycle/        # start-lab.ps1, stop-lab.ps1, teardown-full.ps1, snapshot-amis.ps1
  backend/
    FailoverPoc.Api/   # ASP.NET Core Web API (Linux container), Dockerfile
    FailoverPoc.Data/  # SqlClient/Dapper access layer
  frontend/
    src/app/topology/ | rw-panel/ | load-test/ | failover-timeline/ | data-status/
```

docs/

```
runbook-ag-bringup.md # manual step-by-step log from Phase 3, written as you go
architecture.md
```

NLB health-check mechanism (why no custom health service is needed)

1. Creating the AG Listener makes it a WSFC Client Access Point cluster resource, owned by whichever node hosts the current primary.
2. `Set-ClusterParameter` sets a `ProbePort` (e.g. 59999) on that resource.
3. Only the node currently owning the listener has `rhs.exe` listening on that port — secondaries don't. This is a built-in WSFC behavior, nothing custom to write.
4. NLB target group: TCP health check on port 59999, forwards actual SQL traffic (1433) only to whichever node passes it.
5. App connection string points at the NLB DNS name, port 1433 — no `MultiSubnetFailover` needed (that's for native multi-IP listener DNS, not applicable here).
6. For reads from specific secondaries in the UI (to show all 3 nodes' state at once), connect directly to each node's private IP with `ApplicationIntent=ReadOnly`, bypassing the NLB.

WSFC + AG bring-up — the highest-risk phase, plan for manual-first-then-codify

There is no drop-in Terraform resource for WSFC/AG configuration — it's all imperative PowerShell/T-SQL. Recommended approach: **do the full sequence manually via RDP the first time**, logging every command into `docs/runbook-ag-bringup.md`, then convert that exact sequence into the `scripts/bootstrap/*.ps1` scripts wrapped as SSM documents for repeatable runs. Sequence:

1. Terraform provisions 3 raw Windows EC2 instances + networking (fully automated)
2. Install Windows Failover Clustering feature per node (user-data / SSM — automated)
3. Install SQL Server Developer Edition via Chocolatey per node (SSM — automated, but budget 15-25 min/node and 1-2 iterations to get package args right)
4. Form the **workgroup** WSFC cluster (`New-Cluster -AdministrativeAccessPoint None`) — script it, but watch the first run live via RDP before trusting it unattended. Needs hosts-file DNS entries between the 3 nodes (no AD DNS available) set up in bootstrap.
5. Generate & exchange self-signed certs between nodes, create DATABASE MASTER KEY + cert-based ENDPOINT on each instance — the fiddliest step (cert thumbprint/expiry mismatches are the top cause of AG endpoint failures); expect manual troubleshooting via RDP+SSMS the first time
6. Enable Always On AG feature, create the AG, add replicas, create the listener — do this first pass manually via SSMS wizard to visually confirm replica sync, then codify what worked
7. Set the listener's `ProbePort` cluster parameter (scripted, safe to automate once listener exists)
8. Terraform creates the NLB/target group/health check (fully automated)

Snapshot AMIs of the 3 nodes immediately once the AG is healthy — this lets a future session relaunch from AMI (skipping steps 2-6 entirely) instead of redoing the bring-up.

Cost-control scripts (`scripts/lifecycle/`)

- **start-lab.ps1** — `terraform apply` / start stopped EC2 instances; if AMIs exist, relaunch from them to skip re-running AG bring-up
- **stop-lab.ps1** — for same-day pauses: stop the 3 EC2 nodes + `terraform destroy -target` the NLB/ALB/Fargate service (the pieces that bill while "stopped"); EBS stays (~\$0.47/day, acceptable for a same-day gap)
- **teardown-full.ps1** — for any gap >1 day: run `snapshot-amis.ps1` first (capture configured-node AMIs), then full `terraform destroy`
- Discipline: run full teardown at the end of every session unless resuming same-day — this is what makes the \$100 last for dozens of sessions

.NET backend (`backend/FailoverPoc.Api`)

- `GET /api/health/db` — pings each of the 3 nodes directly (bypassing NLB), returns role + `sys.dm_hadr_availability_replica_states.synchronization_health` + last redo time
- `GET /api/health/cluster` — derived from the same DMV queries across all 3 nodes (avoids needing WSFC API access from the app tier)
- `POST /api/data/write` (via NLB), `GET /api/data/read` (direct to a chosen secondary, `ApplicationIntent=ReadOnly`) — Dapper or EF Core over `Microsoft.Data.SqlClient`
- `POST /api/loadtest/start` / `/stop` — concurrent read/write load generator, live stats pushed over SignalR
- `POST /api/admin/failover` — supports both planned (`ALTER AVAILABILITY GROUP ... FAILOVER T-SQL`) and unplanned (stop the SQL service/EC2 instance on the current primary via AWS SDK) — toggle in UI
- Polly retry policy (3 retries, exponential backoff) wrapping all DB calls — this is what keeps the app "working" visibly during the NLB failover-detection window; surface retry activity in the UI via SignalR rather than hiding it
- `ClusterHub` (SignalR) pushes role changes, load-test stats, failover timeline events
- Deployed as a Linux container on ECS Fargate behind an ALB

Angular frontend (`frontend/`)

- Topology view: 3 node boxes + listener, live PRIMARY/SECONDARY badges, color-coded health
- Read/write panel, load-test panel (start/stop, concurrency/duration, live throughput/latency/error chart — use the `dataviz` skill when building this)
- Failover timeline: append-only SignalR-driven event log — the centerpiece of the demo
- Data status panel: per-node row counts, last write time, replication lag
- `@microsoft/signalr` client connecting through CloudFront → ALB (WebSocket-enabled)
- Deployed to S3 + CloudFront (`ng build --configuration production` , `aws s3 sync` , CloudFront invalidation)

Build order

1. **Phase 0** (no cost): AWS Budget alerts (\$20/\$50/\$80), scoped IAM user for Terraform, install Terraform/AWS CLI
2. **Phase 1** (near-zero cost): Terraform network module — VPC, 3 public subnets, IGW, route tables, SGs

3. **Phase 2** (cost starts): Terraform sql-nodes module — 3x EC2 Windows t3.large + SSM; verify SSM connectivity, then **stop instances** while building later phases
4. **Phase 3** (highest risk — budget the most time/\$ here): manual-first WSFC+AG bring-up via RDP, documented into the runbook, then codified into scripts/SSM docs; snapshot AMIs once healthy
5. **Phase 4**: Terraform NLB + target group + health check; set ProbePort; validate failover re-routes traffic in ~30-40s via `sqlcmd`
6. **Phase 5**: .NET API built and validated locally against the live NLB endpoint, then containerized and deployed to ECS Fargate + ALB
7. **Phase 6**: Angular UI built against the deployed API, deployed to S3 + CloudFront, verify SignalR over WebSocket end-to-end
8. **Phase 7**: load-test + failover-trigger + timeline features wired up; full rehearsal (start load test → trigger failover → confirm UI shows retry/recovery live)
9. **Phase 8**: finalize and dry-run the start/stop/teardown/restore-from-AMI scripts before relying on them

Phase 3 is by far the riskiest and most iteration-prone step (workgroup cluster + cert-based endpoint auth); Phase 4 is second riskiest since it depends on Phase 3's listener resource existing. Phases 5 onward are standard, well-documented .NET/Angular/AWS work.

Verification

- Phase 4: `sqlcmd -S <nlb-dns>,1433 -Q "SELECT @@SERVERNAME"` repeatedly during a manual failover, confirm it always returns the current primary's name within ~30-40s of a failover
- Phase 5: hit `/api/health/db` and `/api/health/cluster` against the live cluster, confirm roles match SSMS
- Phase 7: run the full demo loop (start load test, trigger failover, watch UI) end-to-end before considering the POC done
- Throughout: after every session, confirm via AWS Cost Explorer / Billing console that no unexpected resources are still running